

ARCHITECT-LEVEL DEEP DIVE

Swift

Concurrency: The Complete Guide

From threading nightmares to structured concurrency — every concept, every production bug, every pattern you need to know



async/await



Structured Concurrency



Actors



Task Groups



Sendable



Production Bugs

Swift 5.5+ · Xcode 13+ · iOS 15+ / macOS 12+ · May 2026

Table of Contents

01 The Concurrency Problem — Why It Matters

- What is concurrency and why is it hard?
- Race conditions, deadlocks, priority inversion
- Thread explosion problem

02 The Old World — Threads, GCD, and Callbacks

- NSThread, OperationQueue, Grand Central Dispatch
- Completion handlers and callback hell
- Why GCD is insufficient

03 Why Structured Concurrency Was Needed

- The 10 fundamental problems with unstructured concurrency
- The structured concurrency guarantee

04 `async/await` — The Foundation

- Suspension points, continuations, thread hopping
- Under the hood: cooperative thread pool

05 Tasks — Units of Concurrent Work

- Task, detached tasks, task hierarchy
- Task cancellation, priority, local values

06 TaskGroup — Structured Parallelism

- `withTaskGroup` and `withThrowingTaskGroup`
- Dynamic fan-out patterns

07 `async let` — Concurrent Binding

08 Actors — Safe Shared State

- Actor isolation, reentrancy, actor hopping
- Global actors and `@MainActor`

09 Sendable — The Concurrency Safety Type System

10 AsyncSequence and AsyncStream

11 Continuations — Bridging Old APIs

12 Production Issues, Pitfalls & Real Bugs

- 12 common production bugs with diagnosis and fixes

13 Architectural Patterns & Best Practices

14 Quick Reference Cheat Sheet

The Concurrency Problem

Modern apps are inherently concurrent. They download data, parse JSON, update the UI, play audio, and respond to user input — all at the same time. Getting this right without crashing, hanging, or silently corrupting data is one of the hardest problems in software engineering.

What Is Concurrency?

Concurrency is the ability of a program to make progress on multiple tasks at the same time. This is distinct from **parallelism**, where tasks literally execute simultaneously on multiple CPU cores.



Concurrency

Multiple tasks make progress, but may not run at the exact same instant. The scheduler interleaves them. Think: single-core multitasking.



Parallelism

Multiple tasks literally execute at the same instant on different CPU cores. True simultaneous execution. Requires multiple cores.



Synchronization

Coordinating access to shared resources so concurrent tasks don't corrupt each other's data. Locks, queues, actors.



Async/Await

A language-level mechanism that allows a function to *suspend* without blocking a thread, resuming later when ready.

The Core Problems

Concurrent programming has three foundational failure modes that have caused countless production crashes:

1

Race Condition

Two or more threads read and write shared data concurrently. The outcome depends on the unpredictable scheduling order. The bug appears intermittently, making it nearly impossible to reproduce in testing. Example: two threads both read a counter, both increment it, and write back — but one increment is lost.

2

Deadlock

Thread A holds Lock 1 and waits for Lock 2. Thread B holds Lock 2 and waits for Lock 1. Both wait forever. The app freezes silently. GCD's `sync` on the same queue from within that queue is the most common iOS deadlock.

3

Priority Inversion

A high-priority thread waits for a resource held by a low-priority thread. The system keeps the low-priority thread from running (scheduling other medium-priority threads), causing the high-priority thread to starve. Famous example: Mars Pathfinder mission in 1997.

4

Thread Explosion

Each blocking operation needs its own thread. Under high concurrency, the system spawns hundreds of threads. Each thread uses ~1MB of stack memory and expensive context-switching. iOS limits ~64 threads per process; hitting this causes queue suspension or crashes.

Thread Explosion Visualized

The following diagram shows what happens when 64 network requests are made simultaneously using blocking GCD vs. Swift concurrency:

GCD APPROACH — Each request blocks a thread (64 threads created):

Thread-1	setup	waiting for network...	parse
Thread-2	setup	waiting for network...	parse
Thread-3	setup	waiting for network...	parse

... (61 more
threads) ...

SWIFT CONCURRENCY – All requests on 4 cooperative threads (CPU core count):

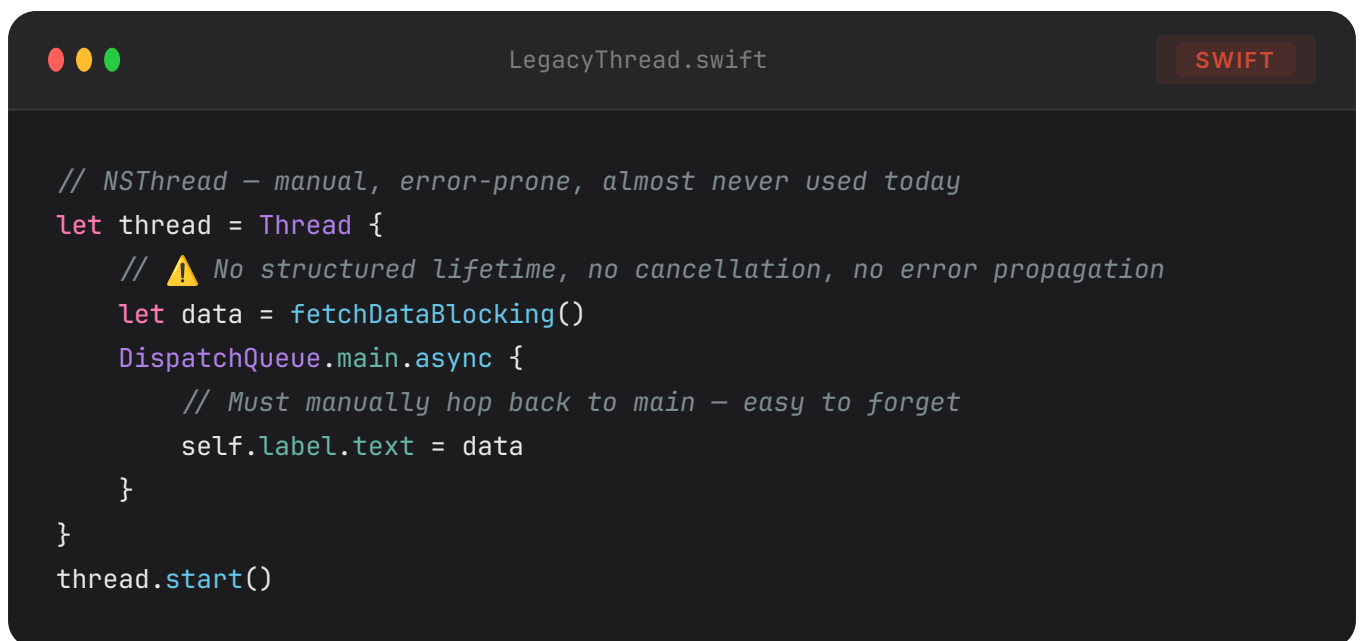
Coop-Thread-1	req1	.	req5	.	req9
Coop-Thread-2	req2	.	req6	.	req1
Coop-Thread-3	req3	.	req7	.	req1
Coop-Thread-4	req4	.	req8	.	req1

The Old World: Threads, GCD & Callbacks

Before Swift 5.5, iOS developers used Grand Central Dispatch, NSThread, OperationQueue, and completion handlers to write concurrent code. Each approach solved some problems while introducing new ones.

NSThread — Manual Thread Management

The earliest approach. You create and manage threads explicitly. This is rarely used in modern iOS development, but understanding it is foundational.



```
// NSThread - manual, error-prone, almost never used today
let thread = Thread {
    // ⚠ No structured lifetime, no cancellation, no error propagation
    let data = fetchDataBlocking()
    DispatchQueue.main.async {
        // Must manually hop back to main - easy to forget
        self.label.text = data
    }
}
thread.start()
```

Grand Central Dispatch (GCD)

GCD abstracts thread management behind a queue model. You submit *work items* to queues, and the system manages threads. GCD was revolutionary in 2009 and still underpins Swift concurrency internally.



```
// — GCD Queue Types —
```

```

// 1. Main Queue (serial, runs on main thread)
DispatchQueue.main.async {
    myLabel.text = "Updated"
}

// 2. Global Concurrent Queue (system-managed thread pool)
DispatchQueue.global(qos: .userInitiated).async {
    let result = expensiveComputation()
    DispatchQueue.main.async {
        self.updateUI(result)
    }
}

// 3. Custom Serial Queue (protects shared state)
let cacheQueue = DispatchQueue(label: "com.app.cache")
var cache: [String: Data] = [:]

func setCache(_ data: Data, key: String) {
    cacheQueue.async { // serialize writes
        cache[key] = data
    }
}

func getCache(key: String) → Data? {
    cacheQueue.sync { // ⚠ blocks current thread!
        return cache[key]
    }
}

// 4. Concurrent Queue with Barrier (reader-writer pattern)
let rwQueue = DispatchQueue(label: "com.app.rw",
                             attributes: .concurrent)

func write(_ value: Int) {
    rwQueue.async(flags: .barrier) {
        sharedData = value // exclusive write
    }
}

func read() → Int {
    rwQueue.sync { sharedData } // concurrent read
}

```

Callback Hell — The Real Problem

Real apps have multi-step asynchronous operations. With callbacks, this leads to deeply nested, unreadable, unmaintainable code:

```
// 🤖 Real-world callback hell: login → fetch profile → fetch posts → update UI
func loadUserDashboard(userId: String) {
    authenticate(userId: userId) { result in
        switch result {
        case .failure(let error):
            DispatchQueue.main.async { self.showError(error) } // easy to forget
        case .success(let token):
            self.fetchProfile(token: token) { profile in
                guard let profile = profile else {
                    DispatchQueue.main.async { self.showError(.missingProfile) }
                    return
                }
                self.fetchPosts(userId: profile.id) { posts, error in
                    if let error = error {
                        DispatchQueue.main.async { self.showError(error) }
                        return
                    }
                    fetchRecommendations(for: profile) { recommendations in
                        DispatchQueue.main.async {
                            // Are we even on the right thread here? Who knows.
                            self.updateUI(profile: profile,
                                           posts: posts ?? [],
                                           recommendations: recommendations ?? [])
                        }
                    }
                }
            }
        }
    }
}
```



The 7 Sins of Callback-Based Concurrency

1. **Pyramid of doom** — deeply nested, hard to read
2. **Error handling duplication** — each level must handle errors
3. **Thread-hopping is manual** — easy to update UI on background thread
4. **No cancellation support** — work continues even when the view is gone
5. **No structured lifetime** — callbacks can outlive the object that created them
6. **Impossible to reason about** — control flow is non-linear
7. **Retain cycle risk** — closures capturing `self` must use `[weak self]`

Why Structured Concurrency Was Needed

Swift's structured concurrency (SE-0296, SE-0304, introduced in Swift 5.5) was designed to fundamentally change how we think about concurrent code — making it as safe and readable as sequential code.

The Core Insight: Control Flow Matters

Structured programming (1960s) introduced the insight that goto statements make code impossible to reason about, and gave us `if`, `for`, `while`, and function calls — structured control flow. The same insight applies to concurrency.

The Structured Concurrency Guarantee

In structured concurrency, a task's **lifetime is always nested within the scope that created it**. When a scope exits, all tasks it spawned are either completed or cancelled. This is the same guarantee structured programming gives you for loops and functions — you always know when they start and end.

10 Problems Structured Concurrency Solves

✗ Problem 1: Unstructured Lifetime

GCD tasks can outlive the object that created them. A `DispatchQueue.async` block runs even after `self` is deallocated.

✓ Solution: Scoped Tasks

Child tasks are automatically cancelled when their parent scope exits. No lingering callbacks holding onto deallocated objects.

✗ Problem 2: No Automatic Cancellation

With GCD, if a user navigates away, in-flight requests keep running, wasting network and battery, and potentially crashing on callback.

✓ Solution: Hierarchical Cancellation

Cancel a parent task and all child tasks are cancelled automatically. Tasks check `Task.isCancelled` at suspension points.

✗ Problem 3: Error Propagation is Manual

In nested callbacks, every level must explicitly handle and forward errors. Miss one and the error is silently swallowed.

✓ Solution: async throws

Errors thrown in async functions propagate up the call stack just like in synchronous code, using standard `try/catch`.

✗ Problem 4: Thread Explosion

Blocking GCD threads while waiting for I/O causes the system to spawn more threads, leading to hundreds of threads fighting for CPU time.

✓ Solution: Cooperative Suspension

async/await suspends the task (not the thread) while waiting. The thread is freed to do other work. Thread count stays $\approx$ CPU core count.

✗ Problem 5: Data Races

Nothing prevents concurrent access to shared mutable state. Race conditions are common and nearly impossible to detect with static analysis.

✓ Solution: Actors + Sendable

The Actor model serializes access to mutable state. The Sendable protocol lets the compiler statically detect unsafe data sharing.

✗ Problem 6: Priority Inversion

Mixing QoS levels in GCD easily leads to priority inversion, where high-priority work starves waiting for a low-priority lock holder.

✓ Solution: Priority Propagation

Swift concurrency automatically propagates task priority. When a high-priority task awaits something, it boosts the priority of the awaited work.

❌ Problem 7: Deadlocks

`DispatchQueue.sync` on the current queue, or mutual `sync` calls between queues, causes instant deadlock. No compiler warning.

✅ Solution: No Blocking APIs

Swift concurrency has no `sync` equivalent. All coordination uses `await`, which suspends (non-blocking). Deadlock by blocking is impossible by design.

❌ Problem 8: Impossible to Reason About

Callback-based code has non-linear control flow. You can't read it top-to-bottom. Stack traces are useless. Debugging requires mental simulation.

✅ Solution: Linear-Looking Code

async/await makes async code look and read like synchronous code. Control flow is linear. Stack traces include async frames. Debugger works naturally.

❌ Problem 9: Retain Cycles

Every GCD callback capturing `self` requires careful `[weak self]` management. Forgetting creates memory leaks, especially in ViewModels.

✅ Solution: Structured Lifetimes

Since task lifetimes are bounded by their scope, there's no cycle — the task holds a reference to the actor/class, not a closure that outlives it.

❌ Problem 10: Testing is Hard

Testing async GCD code requires `XCTestExpectation`, `waitForExpectations`, and careful timeout management. Tests are fragile and slow.

✅ Solution: async Test Functions

XCTest supports `async` test functions natively. `await` works in tests. No expectations needed. Tests are deterministic and readable.

async/await — The Foundation

`async/await` is the cornerstone of Swift concurrency. It provides a way to write asynchronous code that reads like synchronous code, with the compiler enforcing correct usage at every step.

The Basics

```
AsyncBasics.swift  SWIFT

// Marking a function as async means it CAN suspend
func fetchUser(id: String) async throws → User {
    let url = URL(string: "https://api.example.com/users/\(id)")!
    let (data, response) = try await URLSession.shared.data(from: url)
    //           ↑ SUSPENSION POINT: thread is freed here
    //           When response arrives, task RESUMES (possibly on different thread)

    guard (response as? HTTPURLResponse)?.statusCode == 200 else {
        throw APIError.invalidResponse
    }
    return try JSONDecoder().decode(User.self, from: data)
}

// Calling an async function — must use await
func loadProfile() async {
    do {
        let user = try await fetchUser(id: "user-123")
        // Code here runs AFTER the await — sequential, readable
        let posts = try await fetchPosts(for: user)
        let feed = try await buildFeed(user: user, posts: posts)
        display(feed)
    } catch {
        handleError(error) // one place to handle all errors!
    }
}
```

What Really Happens at a Suspension Point

This is the most critical concept to understand. When Swift hits an `await`, the following happens at runtime:

1

Suspend the Task (NOT the Thread)

The async function's execution state (local variables, program counter) is saved to the heap as a "continuation." The thread is immediately freed and returns to the pool.

2

Thread Does Other Work

The freed thread picks up other runnable tasks from the cooperative thread pool. It doesn't block. This is how 1000 concurrent tasks can run on 4 threads.

3

Event Arrives (network, timer, etc.)

The awaited operation completes. The system marks the continuation as "ready to resume" and enqueues it back onto the thread pool.

4

Task Resumes (possibly on a different thread)

An available thread picks up the continuation and resumes execution from exactly where it left off. This is why you must not use thread-local storage across await points.



Important: Thread Identity Across Await

After an `await`, your code may resume on a **different thread** than it started on (unless you're in a `@MainActor` context or global actor). Never use thread-local storage (`pthread_getspecific`) or assume thread affinity across suspension points. This also means `os_unfair_lock` (a non-reentrant lock) is dangerous in async context.

Cooperative Thread Pool — Under the Hood

Swift's concurrency runtime maintains a **cooperative thread pool** whose size is limited to the number of CPU cores (typically 4–8 on an iPhone). This is the architectural reason why Swift concurrency eliminates thread explosion:



ThreadPool.swift

```
// Conceptual model of how the cooperative thread pool works:
// (This is not real Swift code – it illustrates the runtime behavior)

struct CooperativeThreadPool {
    let threadCount = ProcessInfo.processInfo.processorCount // e.g., 4
    var runQueue: [Continuation] = [] // ready-to-run task continuations

    // Each thread runs this loop:
    func workerLoop() {
        while true {
            let continuation = runQueue.pop() // grab next work item
            continuation.resume() // run until next await
            // if continuation hits await, it suspends and thread loops back
            // if continuation completes, thread loops back
        }
    }
}

// CRITICAL: Do NOT block a thread in async context!
func badAsyncFunction() async {
    Thread.sleep(forTimeInterval: 5) // ❌ BLOCKS the thread!
    // This starves the pool: one of 4 threads is blocked for 5 seconds
    // All other tasks on this thread starve
}

func goodAsyncFunction() async {
    try await Task.sleep(for: .seconds(5)) // ✅ SUSPENDS, frees thread
}
```

async let — Concurrent Binding (Sneak Peek)



AsyncLet.swift

SWIFT

```
// Sequential: takes 2 seconds total (1s + 1s)
func loadSequential() async throws → (User, [Post]) {
    let user = try await fetchUser() // waits 1 second
    let posts = try await fetchPosts() // then waits 1 second
    return (user, posts)
}

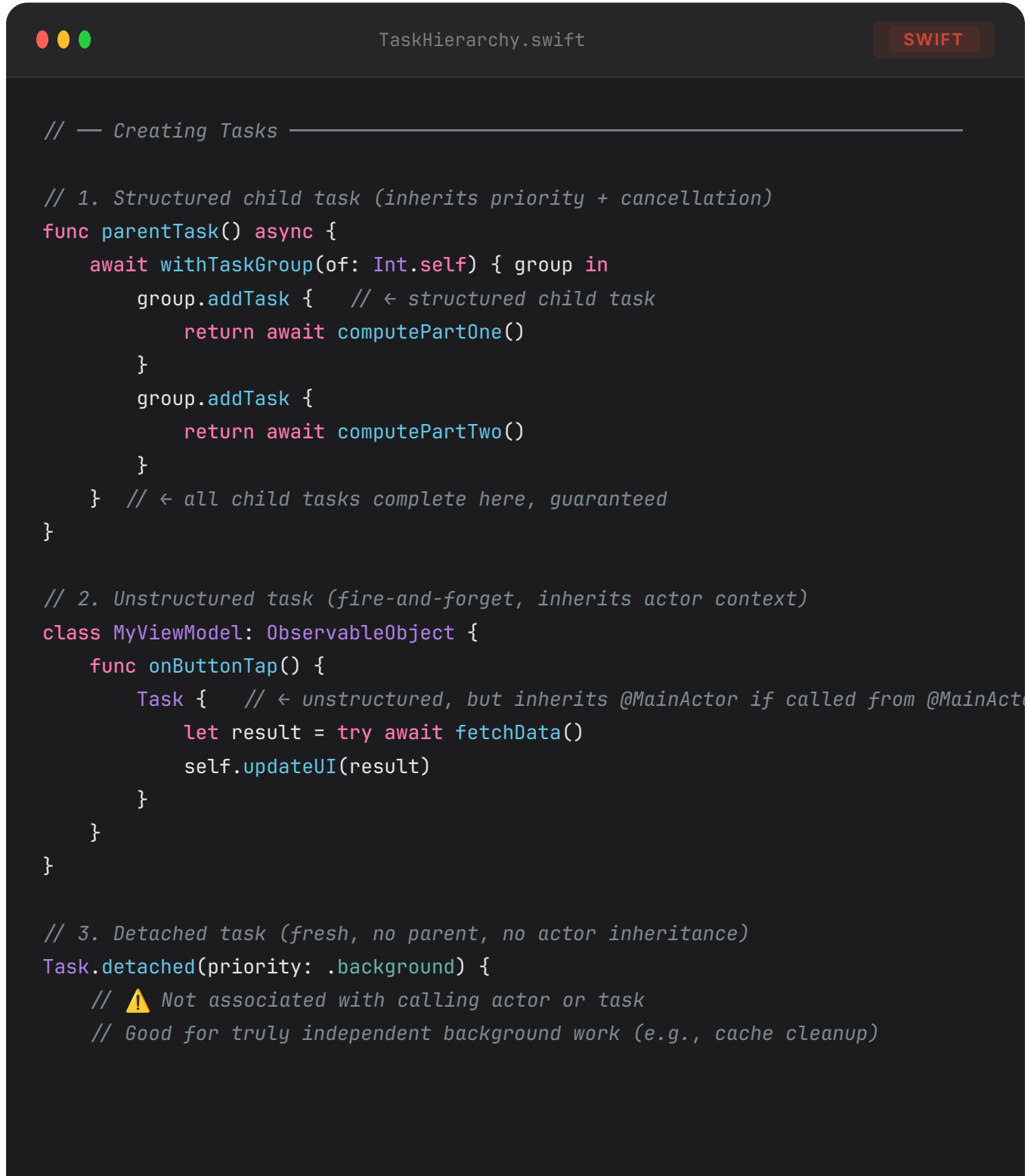
// Concurrent with async let: takes 1 second total (concurrent)
func loadConcurrent() async throws → (User, [Post]) {
    async let user = fetchUser() // starts immediately, does NOT await yet
```

```
async let posts = fetchPosts() // starts immediately, does NOT await yet
// Both are running in parallel NOW
return try await (user, posts) // await both; waits for slower one
}
```

Tasks — Units of Concurrent Work

A **Task** is the fundamental unit of asynchronous work in Swift. Every piece of async code runs inside a task. Understanding the task hierarchy is essential to understanding structured concurrency.

Task Hierarchy



The screenshot shows a Swift code editor window titled "TaskHierarchy.swift" with a "SWIFT" language indicator. The code is organized into three sections, each starting with a comment: "Creating Tasks", "Structured child task", and "Unstructured task".

```
// — Creating Tasks —————

// 1. Structured child task (inherits priority + cancellation)
func parentTask() async {
    await withTaskGroup(of: Int.self) { group in
        group.addTask { // ← structured child task
            return await computePartOne()
        }
        group.addTask {
            return await computePartTwo()
        }
    } // ← all child tasks complete here, guaranteed
}

// 2. Unstructured task (fire-and-forget, inherits actor context)
class MyViewModel: ObservableObject {
    func onTap() {
        Task { // ← unstructured, but inherits @MainActor if called from @MainActor
            let result = try await fetchData()
            self.updateUI(result)
        }
    }
}

// 3. Detached task (fresh, no parent, no actor inheritance)
Task.detached(priority: .background) {
    // ⚠ Not associated with calling actor or task
    // Good for truly independent background work (e.g., cache cleanup)
```

```
    await cleanupOldCacheFiles()
}
```

Task Cancellation

Cancellation in Swift concurrency is **cooperative**: cancelling a task sets a flag, but the task must check and respond to it.

```
TaskCancellation.swift  SWIFT

// — Cancellation Patterns —————

// Store a Task reference to cancel it later
class SearchViewController: UIViewController {
    private var searchTask: Task<Void, Never>?

    func searchTextChanged(_ text: String) {
        searchTask?.cancel() // cancel previous search
        searchTask = Task {
            try await Task.sleep(for: .milliseconds(300)) // debounce
            try Task.checkCancellation() // throws CancellationError if cancelled
            let results = try await performSearch(text)
            await updateSearchResults(results)
        }
    }
}

// Manual cancellation check in CPU-intensive work
func processLargeDataset(_ data: [Record]) async throws → [Result] {
    var results: [Result] = []
    for (index, record) in data.enumerated() {
        if index.isMultiple(of: 100) {
            try Task.checkCancellation() // check every 100 items
        }
        results.append(process(record))
    }
    return results
}

// withTaskCancellationHandler — run cleanup when cancelled
func fetchWithCleanup() async throws → Data {
    try await withTaskCancellationHandler {
        try await performNetworkRequest()
    } onCancel: {
        // Called on cancellation (may be from different thread!)
        networkHandle.cancel() // cancel underlying URLSessionTask
    }
}
```

```
}  
}
```

Task Priority

Priority	Use Case	GCD QoS Equivalent	Notes
<code>.userInteractive</code>	Animations, direct user interaction	<code>userInteractive</code>	Runs on main thread
<code>.userInitiated</code>	User-triggered actions (button tap)	<code>userInitiated</code>	Results needed now
<code>.medium</code>	Default for unstructured tasks	<code>default</code>	No explicit priority set
<code>.utility</code>	Long-running work with visible progress	<code>utility</code>	Responsive but not urgent
<code>.background</code>	Sync, maintenance, invisible work	<code>background</code>	May be throttled heavily
<code>.low</code>	Speculative, preload, unimportant	<code>background</code>	Runs when system is idle

Task Local Values

Task local values are stored on the task itself (not thread-local storage) and are automatically propagated to child tasks:

```
TaskLocals.swift  
  
// Declare task-local values with @TaskLocal  
enum RequestContext {  
    @TaskLocal static var traceID: String? = nil  
    @TaskLocal static var userID: Int? = nil  
}  
  
// Bind task locals for a scope  
func handleRequest() async {
```

SWIFT

```
let traceID = UUID().uuidString
await RequestContext.$traceID.withValue(traceID) {
    // This value is visible to ALL child tasks and awaited functions
    await processRequest()
}

// Deep in the call stack, read the trace ID without passing it explicitly
func logEvent(_ event: String) async {
    let trace = RequestContext.traceID ?? "unknown"
    print("[\($trace)] \($event)") // propagated automatically!
}
```

TaskGroup — Structured Parallelism

TaskGroup is the primary mechanism for running a dynamic number of tasks in parallel and collecting their results. It provides structured concurrency guarantees: all child tasks complete before the group exits.

Basic TaskGroup



TaskGroup.swift

SWIFT

```
// — Basic TaskGroup: Parallel image downloads —————
func downloadImages(urls: [URL]) async throws → [UIImage] {
    try await withThrowingTaskGroup(of: (Int, UIImage).self) { group in
        // Fan out: create one child task per URL
        for (index, url) in urls.enumerated() {
            group.addTask {
                let image = try await downloadImage(from: url)
                return (index, image) // include index to restore order
            }
        }
        // Fan in: collect results as they arrive (in completion order)
        var results = Array<(Int, UIImage)?>(repeating: nil, count: urls.count)
        for try await (index, image) in group { // AsyncSequence iteration
            results[index] = (index, image)
        }
        return results.compactMap { $0 }.sorted { $0.0 < $1.0 }.map { $0.1 }
    }
}

// — Bounded concurrency: limit to N concurrent tasks —————
func processInBatches<T>(items: [T], concurrency: Int) async throws {
    try await withThrowingTaskGroup(of: Void.self) { group in
        var inFlight = 0
        for item in items {
            if inFlight ≥ concurrency {
                try await group.next() // wait for one to finish before adding more
                inFlight -= 1
            }
            group.addTask { try await self.process(item) }
        }
    }
}
```

```

        inFlight += 1
    }
    try await group.waitForAll()
}
}

// — Early exit: first success wins —————
func fetchFromFastestServer(urls: [URL]) async throws → Data {
    try await withThrowingTaskGroup(of: Data.self) { group in
        for url in urls {
            group.addTask { try await URLSession.shared.data(from: url).0 }
        }
        guard let first = try await group.next() else {
            throw URLError(.badURL)
        }
        group.cancelAll() // cancel remaining downloads
        return first
    }
}

```



TaskGroup vs async let

Use `async let` when you have a **fixed, small number** of concurrent operations (known at compile time). Use `TaskGroup` when you have a **dynamic number** of operations (known at runtime, e.g., iterating an array). `TaskGroup` also provides result collection and cancellation controls that `async let` doesn't offer.

async let — Concurrent Binding

`async let` is syntactic sugar for launching a child task and binding its future result to a constant. It's the cleanest way to run a small, fixed number of operations concurrently.

```
AsyncLet.swift SWIFT

// — async let: Full Pattern —————
func loadDashboard() async throws → Dashboard {
    // All three start IMMEDIATELY and run concurrently
    async let user: User = fetchUser()
    async let notifications: [Notification] = fetchNotifications()
    async let feed: [Post] = fetchFeed()

    // Await all three — waits for the SLOWEST one
    return try await Dashboard(
        user: user,
        notifications: notifications,
        feed: feed
    )
    // If ANY throws, the others are cancelled automatically!
}

// — Important: async let creates a child task — must be awaited —————
func example() async throws {
    async let value = heavyComputation()
    // If we leave scope without awaiting, Swift cancels the child task
    // and awaits its cancellation before exiting scope
    if someCondition {
        return // Swift automatically cancels + awaits `value` here
    }
    let result = try await value
    use(result)
}

// — Combining async let with withTaskGroup —————
func loadWithOptionalFeatures() async → AppState {
    async let core: CoreData = loadCoreData()
    async let features: [Feature] = loadFeatureFlags()

    let experiments: [Experiment]
    if isUserLoggedIn {
```

```

        experiments = (try? await loadExperiments()) ?? []
    } else {
        experiments = []
    }
    return await AppState(core: core,
                          features: features,
                          experiments: experiments)
}

```

Deep Dive: Two Critical Questions About async let

Q1 — Can you collect individual results as they arrive, or update incrementally?

No. `async let` **does not stream results**. Each binding is a future — a promise of a single value. You can only read it once, and only by awaiting it. You cannot observe partial progress, react as each one finishes, or process them in completion order.



`async let` Has No "First to Finish" Semantic

With `async let`, even if API-A finishes in 0.1s and API-B takes 3s, you cannot act on API-A's result until you explicitly `await` it. And if you await API-B first, you sit idle for 3 seconds even though API-A's result is already sitting there, done.

`async let` is *parallel execution, serial collection*. The order you `await` them is the order you unblock — not the order they finish.



AsyncLetOrdering.swift

SWIFT

// — Scenario: API-A takes 0.5s, API-B takes 3s —————

```

async let fast = fetchAPIa()    // starts immediately — done in 0.5s
async let slow = fetchAPIb()    // starts immediately — done in 3s

```

// ❌ Case 1: You await slow FIRST

```

let b = try await slow          // blocks here for 3 full seconds
let a = try await fast          // fast was ready 2.5s ago — we just wasted time
updateUI(a: a, b: b)           // UI updates only at t=3s

```

// ❌ Case 2: You want to update UI with fast result immediately

```

let a = try await fast          // unblocks at t=0.5s ✓
updateUIWithPartialData(a)      // ← this is VALID syntax, but...
let b = try await slow          // ...slow was already running in parallel all along

```

```

updateUIWithFullData(a: a, b: b) // unblocks at t=3s
// This DOES work – but only if you know which one is faster at compile time!
// In real apps the order varies per network condition – not reliable.

// ✅ Awaiting individually IS valid for simple "known order" cases:
async let profileImage = fetchAvatar()
async let profileData = fetchProfile() // both run in parallel
let data = try await profileData // show skeleton → data first
showProfile(data) // partial UI update ✓
let image = try await profileImage // then image when ready
showAvatar(image) // complete UI update ✓

```

When You Actually Need "First to Finish / Incremental Updates" — Use TaskGroup

TaskGroup iterates results in *completion order* — whichever child task finishes first is the first value you receive in the **for await** loop. This is the right tool when you want to update UI progressively or act on each result as it arrives.



TaskGroupStreaming.swift

SWIFT

```

// — TaskGroup: process each result the moment it arrives —————
func loadDashboardProgressively() async throws {
    try await withThrowingTaskGroup(of: DashboardSection.self) { group in

        group.addTask { .profile(try await fetchProfile()) } // 0.3s
        group.addTask { .feed(try await fetchFeed()) } // 1.2s
        group.addTask { .ads(try await fetchAds()) } // 2.5s

        // Results arrive in COMPLETION ORDER – not the order added above
        for try await section in group {
            switch section {
            case .profile(let p): await showProfile(p) // fires at t=0.3s ✓
            case .feed(let f): await showFeed(f) // fires at t=1.2s ✓
            case .ads(let a): await showAds(a) // fires at t=2.5s ✓
            }
            // Each case updates the UI immediately when its data is ready
        }
    }
}

```

```

// — async let version: ALL sections only shown at t=2.5s (slowest) —————
func loadDashboardAllAtOnce() async throws {
    async let profile = fetchProfile() // 0.3s – waits in silence
    async let feed = fetchFeed() // 1.2s – waits in silence
    async let ads = fetchAds() // 2.5s – bottleneck
    let (p, f, a) = try await (profile, feed, ads)
}

```

```
showAll(p, f, a) // UI only updates at t=2.5s – no progressive loading
}
```

Capability	async let	TaskGroup
Runs tasks in parallel	Yes	Yes
Fixed number of tasks at compile time	Yes (preferred)	Possible but verbose
Dynamic number of tasks at runtime	No	Yes (its strength)
Process results in completion order	No — await order only	Yes — for await loop
Incremental / progressive UI updates	Only if you know fastest first	Yes — native
Cancel all if one fails	Automatic	Automatic (throwing variant)
Cancel some, keep others	No	Yes — group.cancelAll()
Collect partial results before error	No	Yes — iterate before failure

Q2 — What happens if one of the async let tasks throws?

This is one of the most important behaviors to understand. The rules are precise and were deliberately designed this way:



The Three-Phase Error Rule for async let

Phase 1 — Tasks run in parallel. Both tasks execute concurrently. If one throws, it does not immediately stop the other — the error is stored and held.

Phase 2 — Error surfaces at the await point. The error is only thrown when you attempt to `await` the binding that failed. Code between the `async let` declaration and the `await` continues normally.

Phase 3 — Other bindings are automatically cancelled. When the error propagates out of the scope (i.e., the function throws), Swift automatically cancels any `async let` bindings that haven't been awaited yet and waits for them to finish cancelling before the scope exits.

```
// — Scenario A: One throws, you await the throwing one first —————
func loadDashboard() async throws → Dashboard {
    async let user = fetchUser()           // ← will throw NetworkError
    async let posts = fetchPosts()         // ← will succeed
    // Both tasks are running in parallel right now

    let u = try await user                // ← throws NetworkError here
    // Execution stops. The `posts` child task is CANCELLED automatically.
    // Swift awaits posts cancellation, then propagates the error to the caller.
    let p = try await posts               // ← NEVER REACHED
    return Dashboard(user: u, posts: p)
}
// Caller receives: throw NetworkError

// — Scenario B: One throws, you await the NON-throwing one first —————
func loadDashboardB() async throws → Dashboard {
    async let user = fetchUser()           // ← will throw NetworkError
    async let posts = fetchPosts()         // ← will succeed

    let p = try await posts               // ← succeeds, returns [Post]
    // posts result is in hand. user task may have ALREADY thrown by now,
    // but the error is held – not yet surfaced.
    let u = try await user                // ← throws NetworkError HERE (error surfaces now)
    // `p` was fetched and computed, but it's DISCARDED – function throws anyway.
    return Dashboard(user: u, posts: p)
}
// posts work was wasted. NetworkError is thrown. p is gone.

// — Scenario C: BOTH throw —————
func loadDashboardC() async throws → Dashboard {
    async let user = fetchUser()           // ← throws NetworkError.timeout
    async let posts = fetchPosts()         // ← throws NetworkError.serverError

    let u = try await user                // ← throws NetworkError.timeout
    // posts is cancelled. NetworkError.timeout propagates.
    // NetworkError.serverError from posts is SILENTLY DROPPED.
    let p = try await posts               // ← NEVER REACHED
    return Dashboard(user: u, posts: p)
}
// Only FIRST awaited error is thrown. The other error is lost.
```



Critical: When Both Throw, Only One Error Survives

If both `async let` bindings throw, you only get the error from the one you `await` first. The other error is silently discarded. There is no mechanism in `async let` to collect multiple errors simultaneously. If you need to know about all failures, use `TaskGroup` with manual error collection.

```
AsyncLetErrorStrategies.swift SWIFT

// — Strategy 1: Make errors optional – tolerate individual failures —————
func loadDashboardTolerant() async → Dashboard {
    async let user = try? fetchUser()    // User? – nil on failure
    async let posts = try? fetchPosts()  // [Post]? – nil on failure
    async let ads = try? fetchAds()      // [Ad]? – nil on failure

    // None of these can throw – all errors converted to nil
    return await Dashboard(
        user: user ?? .guest,
        posts: posts ?? [],
        ads: ads ?? []
    )
    // If fetchUser fails, show guest profile. Other sections still load.
    // ✅ Resilient: partial success is useful and shown to the user.
}

// — Strategy 2: Provide fallback values inline —————
async let config = (try? fetchRemoteConfig()) ?? Config.default
async let flags = (try? fetchFeatureFlags()) ?? []
let (c, f) = await (config, flags) // never throws, always has values

// — Strategy 3: Collect ALL errors – use TaskGroup instead —————
func loadWithAllErrors() async → (Dashboard?, [Error]) {
    var errors: [Error] = []
    var user: User?
    var posts: [Post]?

    await withTaskGroup(of: Void.self) { group in
        group.addTask {
            do { user = try await fetchUser() }
            catch { errors.append(error) } // capture, don't throw
        }
        group.addTask {
            do { posts = try await fetchPosts() }
            catch { errors.append(error) }
        }
    }
    await group.waitForAll() // non-throwing – errors are already captured
```

```

    }
    guard errors.isEmpty, let u = user, let p = posts else {
        return (nil, errors) // return ALL errors to caller
    }
    return (Dashboard(user: u, posts: p), [])
}

// — Strategy 4: Timeout a single async let —————
func fetchWithTimeout() async throws → Data {
    async let data = fetchData()
    async let timeout: Void = {
        try await Task.sleep(for: .seconds(5))
        throw TimeoutError()
    }()

    // Race: whichever throws or returns first wins
    do {
        let result = try await data
        return result // data won — timeout child task is cancelled
    } catch is TimeoutError {
        throw TimeoutError() // timeout won — data child task is cancelled
    }
}

```

The Complete Error Behaviour Matrix

Scenario	What Actually Happens	What Caller Sees
A throws, B succeeds — await A first	A's error surfaces immediately. B is cancelled. B's result is discarded.	A's error thrown
A throws, B succeeds — await B first	B succeeds and result is in hand. Then awaiting A surfaces its error. B's result is discarded.	A's error thrown (B's work wasted)
Both throw — await A first	A's error surfaces. B is cancelled. B's error is silently dropped.	Only A's error. B's error lost.
Both throw — await B first	B's error surfaces. A is cancelled. A's error is silently dropped.	Only B's error. A's error lost.
Neither throws	Both complete. Results available when awaited.	Both results returned

A throws, use <code>try?</code> on A	A's error is converted to nil. B continues unaffected.	nil for A, B's result — no throw
A throws, use <code>try?</code> on both	Both errors become nil. No throwing, no cancellation.	Both results as optionals
A throws, Task is cancelled externally	CancellationError surfaces. All async let tasks are cancelled.	CancellationError thrown



Design Rule of Thumb

Use `try await` (throwing) on an `async let` only when *all-or-nothing* is the correct semantic — if any one piece fails, the whole operation should fail.

Use `try?` or `try? ... ?? default` when the piece is optional or has a sensible fallback — partial results are still useful to the user.

Use `TaskGroup` **with manual error collection** when you need to report multiple failures simultaneously (e.g., a batch validation screen listing all errors at once).

Actors — Safe Shared State

Actors are Swift's solution to the data race problem. An actor is a reference type that serializes all access to its mutable state. No two callers can access an actor's state simultaneously — the runtime guarantees this at compile time.

The Actor Model

Actors have a private "mailbox" — a serial queue of work items. All external calls to actor methods are enqueued and processed one at a time. From outside the actor, all property accesses and method calls must use `await`.



ActorBasics.swift

SWIFT

```
// — Actor Declaration —————
actor BankAccount {
    private var balance: Double
    private var transactions: [Transaction] = []

    init(initialBalance: Double) {
        self.balance = initialBalance
    }

    // Synchronous inside the actor — no await needed internally
    func deposit(_ amount: Double) {
        balance += amount
        transactions.append(.deposit(amount))
    }

    func withdraw(_ amount: Double) throws {
        guard balance ≥ amount else { throw BankError.insufficientFunds }
        balance -= amount
        transactions.append(.withdrawal(amount))
    }

    // nonisolated — can be called without await (only reads immutable/sendable data)
    nonisolated let accountID: String = UUID().uuidString
}

// — Using the actor from outside —————
func performTransactions() async throws {
```

```

let account = BankAccount(initialBalance: 1000)

// Must use await – may need to wait for actor to be available
await account.deposit(500)
try await account.withdraw(200)

// Even concurrent calls are safe – serialized by the actor
async let d1: Void = account.deposit(100)
async let d2: Void = account.deposit(200)
async let w1: Void = account.withdraw(50)
_ = try await (d1, d2, w1) // All safe – actor serializes them
}

```

Actor Reentrancy — The Most Dangerous Gotcha

This is the most important and misunderstood aspect of Swift actors. Actors are **reentrant**: while an actor is awaiting something, another caller can execute code on the actor. This means that state you checked before an `await` may change by the time the `await` returns.



ActorReentrancy.swift

SWIFT

```

actor ImageCache {
    private var cache: [URL: UIImage] = [:]
    private var inProgress: Set<URL> = []

    // ❌ BUGGY: Reentrancy causes duplicate downloads
    func getImageBuggy(url: URL) async throws → UIImage {
        if let cached = cache[url] { return cached }

        // ← SUSPENSION POINT: actor is reentrant here!
        // Another caller can call getImage(url:) for the same URL
        // and also pass the cache check above (cache still empty)
        let image = try await downloadImage(url: url)

        cache[url] = image // downloaded TWICE if two callers race here
        return image
    }

    // ✅ CORRECT: Guard with inProgress set to prevent duplicate downloads
    func getImageCorrect(url: URL) async throws → UIImage {
        if let cached = cache[url] { return cached }
        if inProgress.contains(url) {
            // Another task is downloading it – wait with polling (or use AsyncStream)
            while inProgress.contains(url) {
                await Task.yield() // yield, let other task complete download
            }
        }
    }
}

```

```

    }
    return cache[url]!
}
inProgress.insert(url) // mark as in-progress BEFORE suspension
defer { inProgress.remove(url) }

let image = try await downloadImage(url: url)
cache[url] = image
return image
}
}

```



Actor Reentrancy is THE #1 Actor Bug

The golden rule: **never assume actor state is unchanged across an await point**. Re-validate all assumptions after every `await` inside an actor. This is the primary source of subtle bugs in actor-heavy codebases.

Global Actors

A global actor is a singleton actor that multiple types can be isolated to. The most important global actor is `@MainActor`.



MainActor.swift

SWIFT

```

// — @MainActor: UI Code Safety —————

// Entire class is isolated to the main thread
@MainActor
class ProfileViewModel: ObservableObject {
    @Published var user: User?
    @Published var isLoading = false

    func loadUser() async {
        isLoading = true // ✓ safe — on main actor
        let fetched = try? await fetchUser() // suspends, goes off main
        user = fetched // ✓ safe — resumed on main actor
        isLoading = false
    }
}

// Marking just a function to run on main actor
func updateLabel() async {
    let text = await computeText() // can run anywhere
}

```

```

    await MainActor.run {
        label.text = text //  runs on main thread
    }
}

// Custom global actor (e.g., for a database layer)
@GlobalActor
actor DatabaseActor {
    static let shared = DatabaseActor()
}

@DatabaseActor
class UserRepository {
    func save(_ user: User) async { /* safe - serialized on DatabaseActor */ }
    func fetch(id: String) async → User? { return nil }
}

```

Actor vs Class (When to Use Which)

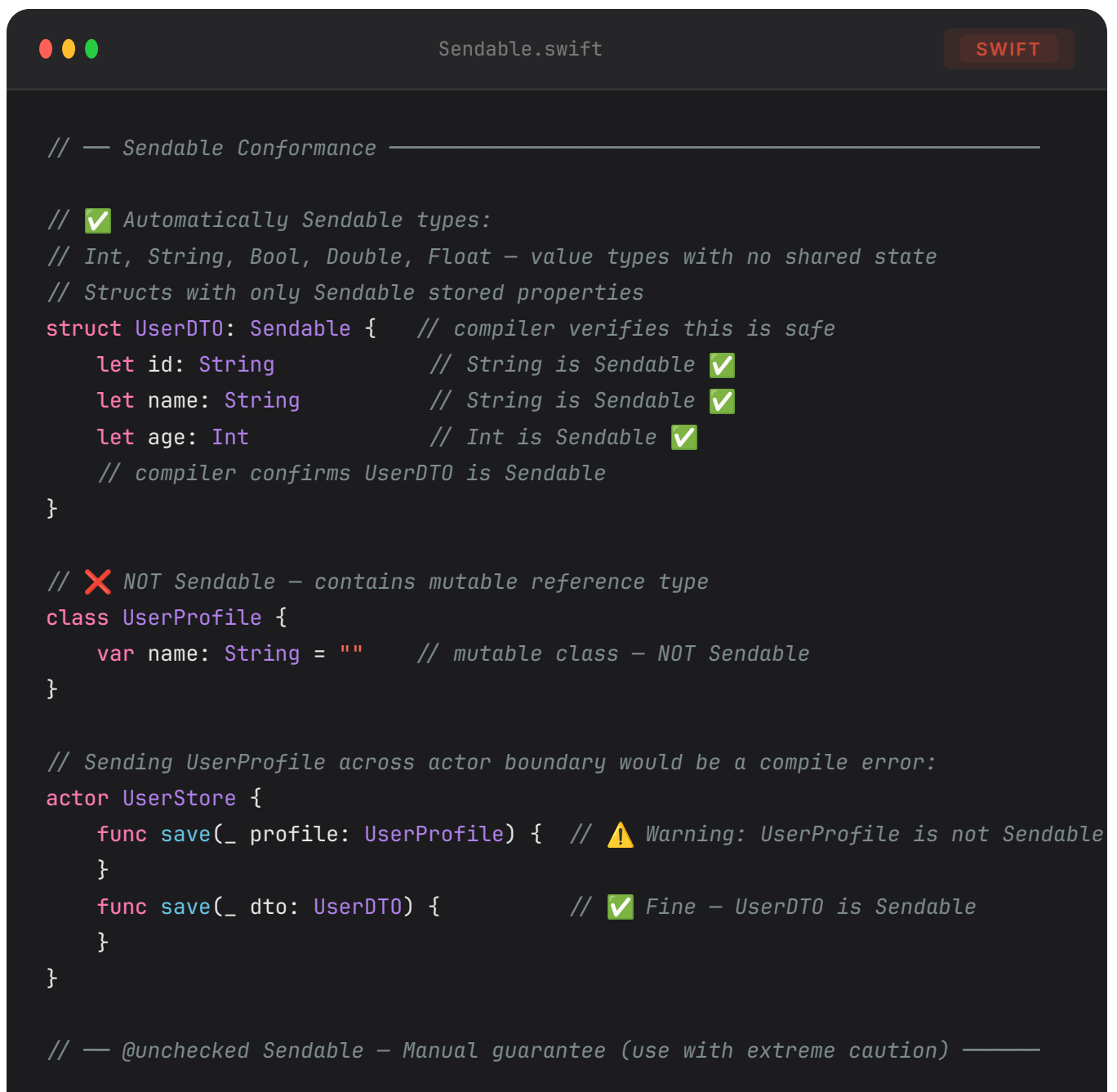
Feature	class	actor	struct
Thread safety	Manual (locks)	Automatic	Value semantics
External property access	Direct	Requires await	Direct
Inheritance	Supported	Not supported	Not supported
Identity	Reference	Reference	Value (copied)
Mutable shared state	Dangerous	Safe	Not shared
Use for UI	With @MainActor	With @MainActor	No

Sendable — The Concurrency Safety Type System

Sendable is a protocol that marks types as safe to share across concurrency domains (actor boundaries, tasks). The Swift compiler enforces this at compile time, making data races detectable before your app ships.

What is Sendable?

A type is **Sendable** if it's safe to transfer between concurrent contexts without risk of data races. The compiler can verify this for many types automatically.



```

Sendable.swift
SWIFT

// — Sendable Conformance —————

// ✅ Automatically Sendable types:
// Int, String, Bool, Double, Float — value types with no shared state
// Structs with only Sendable stored properties
struct UserDTO: Sendable { // compiler verifies this is safe
    let id: String          // String is Sendable ✅
    let name: String        // String is Sendable ✅
    let age: Int             // Int is Sendable ✅
    // compiler confirms UserDTO is Sendable
}

// ❌ NOT Sendable — contains mutable reference type
class UserProfile {
    var name: String = ""    // mutable class — NOT Sendable
}

// Sending UserProfile across actor boundary would be a compile error:
actor UserStore {
    func save(_ profile: UserProfile) { // ⚠️ Warning: UserProfile is not Sendable
    }
    func save(_ dto: UserDTO) {         // ✅ Fine — UserDTO is Sendable
    }
}

// — @unchecked Sendable — Manual guarantee (use with extreme caution) —

```

```

final class ThreadSafeCache: @unchecked Sendable {
    private var storage: [String: Any] = [:]
    private let lock = NSLock()

    func set(_ value: some Sendable, key: String) {
        lock.withLock { storage[key] = value }
    }
    // ⚠️ YOU are responsible for proving thread safety when using @unchecked
}

// — Sendable closures —————
func performOnBackground(_ work: @Sendable () async → Void) async {
    await Task.detached { await work() }.value
}
// @Sendable closure can only capture Sendable values – compiler enforced

```



Strict Concurrency Checking in Swift 6

Swift 6 (available in Xcode 16+) enables **strict concurrency checking by default**. Every data race that the compiler can detect becomes a **compile error**, not just a warning. This makes `Sendable` not just a best practice but a requirement for building with Swift 6. Migrate incrementally using the `-strict-concurrency=targeted` compiler flag.

AsyncSequence & AsyncStream

`AsyncSequence` is the async counterpart to `Sequence`. It produces values over time, allowing you to iterate over them with `for await` just like a regular `for` loop.

`AsyncStream` is the easy way to create your own async sequences.

AsyncSequence Fundamentals

```

AsyncSequence.swift
SWIFT

// — Consuming AsyncSequences —

// URL lines (built-in AsyncSequence)
func readLargeLogs() async throws {
    let url = URL(string: "https://logs.example.com/stream")!
    for try await line in url.lines {
        // Memory efficient: processes one line at a time, no full file in RAM
        processLogLine(line)
    }
}

// Notifications as AsyncSequence
func waitForKeyboardChanges() async {
    let notifications = NotificationCenter.default
        .notifications(named: UIResponder.keyboardWillShowNotification)
    for await notification in notifications {
        adjustLayoutForKeyboard(notification)
        break // stop after first notification (or use Task cancellation)
    }
}

// — Creating AsyncStream —

// Bridge delegate-based API to AsyncStream
func makeLocationStream() → AsyncStream<CLLocation> {
    AsyncStream { continuation in
        let manager = CLLocationManager()
        let delegate = LocationDelegate {
            continuation.yield($0) // emit each location update
        }
        manager.delegate = delegate
    }
}

```

```

manager.startUpdatingLocation()

continuation.onTermination = { _ in
    manager.stopUpdatingLocation() // cleanup when stream ends
}
}

// Use it with for await
func trackUserLocation() async {
    for await location in makeLocationStream() {
        await updateMapAnnotation(location)
        if hasReachedDestination(location) { break }
    }
}

// — AsyncThrowingStream — for error-producing streams —————
func websocketStream(url: URL) → AsyncThrowingStream<Data, Error> {
    AsyncThrowingStream { continuation in
        let socket = openWebSocket(url: url)
        socket.onMessage = { continuation.yield($0) }
        socket.onError = { continuation.finish(throwing: $0) } // propagate error
        socket.onClose = { continuation.finish() } // end stream
    }
}

```

AsyncStream Patterns for Production



```

// — Backpressure with bufferingPolicy —————
func makeThrottledStream() → AsyncStream<Measurement> {
    AsyncStream(bufferingPolicy: .bufferingNewest(10)) { continuation in
        // If consumer is slow, only keep the 10 most recent values
        // older values are dropped (good for real-time sensor data)
        sensor.onReading = { continuation.yield($0) }
    }
}

// — Multicast: share one stream between multiple consumers —————
// (AsyncStream is single-consumer; use actor to multicast)
actor EventBroadcaster<T: Sendable> {
    private var continuations: [UUID: AsyncStream<T>.Continuation] = [:]

    func subscribe() → AsyncStream<T> {
        let id = UUID()

```

```
        return AsyncStream { [weak self] continuation in
            Task { await self?.register(id: id, continuation: continuation) }
            continuation.onTermination = { [weak self] _ in
                Task { await self?.unsubscribe(id: id) }
            }
        }
    }

    func broadcast(_ value: T) {
        continuations.values.forEach { $0.yield(value) }
    }
}
```

Continuations — Bridging Legacy APIs

Continuations allow you to wrap callback-based APIs and present them as async functions. This is your primary bridge between the old world (GCD, delegates, completion handlers) and the new Swift concurrency world.

```
Continuations.swift SWIFT

// — withCheckedContinuation — safe, with runtime checks —————
func fetchData(id: String) async throws → Data {
    return try await withCheckedThrowingContinuation { continuation in
        // Legacy API with completion handler
        LegacyNetworkClient.fetch(id: id) { data, error in
            if let error = error {
                continuation.resume(throwing: error)
            } else if let data = data {
                continuation.resume(returning: data)
            } else {
                continuation.resume(throwing: NetworkError.noData)
            }
        }
        // ⚠ CRITICAL: Must call resume EXACTLY ONCE
        // Calling it 0 times: task leaks forever
        // Calling it 2+ times: crash/undefined behavior
    }
}

// — withUnsafeContinuation — no runtime checks, faster, more dangerous —
func fetchDataFast(id: String) async throws → Data {
    return try await withUnsafeThrowingContinuation { continuation in
        // Same as checked, but no runtime assertions if you misuse it
        // Only use when you've proven correctness and need every nanosecond
        LegacyClient.fetch(id: id, completion: continuation.resume)
    }
}

// — Wrapping a delegate callback —————
class PhotoPicker: NSObject, UIImagePickerControllerDelegate {
    private var continuation: CheckedContinuation<UIImage?, Never>?

    func pickPhoto() async → UIImage? {
```

```

        return await withCheckedContinuation { continuation in
            self.continuation = continuation
            let picker = UIImagePickerController()
            picker.delegate = self
            present(picker, animated: true)
        }
    }

    // UIImagePickerControllerDelegate method
    func imagePickerController(/* ... */, didFinishPickingMediaWithInfo info: [...]) {
        let image = info[.originalImage] as? UIImage
        continuation?.resume(returning: image)
        continuation = nil // nil out to prevent double-resume
    }
}

```



The #1 Continuation Bug: Forgetting to Resume

If the continuation is never resumed (e.g., the callback is never called, or only called on one code path), the awaiting task is **leaked forever**. It will never complete, never be freed, and any resources it holds are never released. Always use `withCheckedContinuation` during development — it will assert if you leak or double-resume. Use `defer` to ensure resume is always called.

Production Issues, Pitfalls & Real Bugs

This chapter catalogs 12 real production bugs that commonly occur when adopting Swift concurrency. Each entry includes the root cause, a minimal reproduction, and the fix.

CRITICAL

Bug #1: Thread Blocking in Cooperative Pool

Calling blocking APIs (file I/O, `Thread.sleep`, `DispatchQueue.sync`, `semaphore.wait`) inside an async function. With 4 threads in the pool, blocking 4 tasks starves all other work in the app.

BROKEN CODE

```
func loadConfig() async → Config {
    let sem = DispatchSemaphore(value: 0)
    var config: Config?
    legacyLoader.load { result in // callback-based API
        config = result
        sem.signal()
    }
    sem.wait() // ❌ BLOCKS thread! Deadlock risk if 4 tasks do this
    return config!
}
```

FIX

```
func loadConfig() async → Config {
    return await withCheckedContinuation { continuation in
        legacyLoader.load { result in
            continuation.resume(returning: result) // ✅ suspends, doesn't block
        }
    }
}
```

CRITICAL

Bug #2: Actor Reentrancy — State Mutation Between Awaits

Assuming actor state is consistent across an `await` inside the actor. After the `await` returns, another task may have modified the state you checked before the `await`.

BROKEN CODE

```
actor OrderManager {
    var pendingOrders: [Order] = []

    func processNextOrder() async {
        guard !pendingOrders.isEmpty else { return }
        let order = pendingOrders.first!           // ← state checked here
        let result = await paymentService.charge(order) // ← SUSPENSION: state may change!
        pendingOrders.removeFirst()                // ✗ might remove wrong order!
        fulfill(result)
    }
}
```

FIX

```
actor OrderManager {
    func processNextOrder() async {
        guard !pendingOrders.isEmpty else { return }
        let order = pendingOrders.removeFirst()    // ✓ Remove BEFORE await
        let result = await paymentService.charge(order)
        fulfill(result)
        // No stale state assumption – order was captured before suspension
    }
}
```

CRITICAL Bug #3: MainActor Deadlock with DispatchQueue.main.sync

Calling `DispatchQueue.main.sync` from an `async` context that is already isolated to `@MainActor`. The `sync` call tries to enqueue work on the main queue but the main actor is already executing, causing a deadlock.

BROKEN CODE

```
@MainActor
func updateUIAndLegacy() async {
    updateSwiftUI()
    DispatchQueue.main.sync {           // ✗ DEADLOCK: already on main actor
        updateLegacyView()             // this can never run while we're blocked here
    }
}
```

FIX

```
@MainActor
func updateUIAndLegacy() async {
    updateSwiftUI()
    updateLegacyView() // ✓ Already on main actor – call directly
}
```

```
// OR use await MainActor.run { } if called from non-MainActor context
}
```

HIGH Bug #4: Task Leak from Uncancelled Long-Running Task

Creating a `Task` in a `UIViewController` or `ViewModel` but never cancelling it in `deinit` or `onDisappear`. The task holds a strong reference to `self`, preventing deallocation.

BROKEN CODE

```
class FeedViewModel {
    func startPolling() {
        Task { // ❌ No reference kept – cannot cancel. Holds strong self.
            while true {
                let feed = try await fetchFeed()
                self.updateFeed(feed)
                try await Task.sleep(for: .seconds(30))
            }
        }
    }
    deinit { /* Task still running! ViewModel leaks */ }
}
```

FIX

```
class FeedViewModel {
    private var pollingTask: Task<Void, Never>?

    func startPolling() {
        pollingTask = Task { [weak self] in // ✅ weak self + stored reference
            while !Task.isCancelled {
                guard let self else { return }
                let feed = try? await self.fetchFeed()
                self.updateFeed(feed)
                try? await Task.sleep(for: .seconds(30))
            }
        }
    }
    deinit { pollingTask?.cancel() } // ✅ clean up on deallocation
}
```

HIGH Bug #5: Forgetting to Check Cancellation in CPU-Bound Loops

Long-running synchronous loops inside async functions don't automatically check cancellation. Even if the parent task is cancelled, the loop runs to completion, wasting CPU and blocking task cleanup.

BROKEN CODE

```
func processAllRecords(_ records: [Record]) async throws → [Result] {
    var results = [Result]()
    for record in records {
        results.append(try processRecord(record)) // ❌ no cancellation check
    }                                           // runs fully even if cancelled
    return results
}
```

FIX

```
func processAllRecords(_ records: [Record]) async throws → [Result] {
    var results = [Result]()
    for (i, record) in records.enumerated() {
        try Task.checkCancellation() // ✅ check periodically
        results.append(try processRecord(record))
        if i % 100 == 0 { await Task.yield() } // ✅ yield for other tasks
    }
    return results
}
```

HIGH Bug #6: UI Update on Wrong Thread (Missing @MainActor)

Updating UI properties after an `await` when the function is not isolated to `@MainActor`. After an `await`, the task may resume on a background thread, and UIKit/AppKit updates from non-main threads cause crashes or visual glitches.

BROKEN CODE

```
class ProductVC: UIViewController {
    func loadProduct(id: String) {
        Task {
            let product = try await fetchProduct(id: id) // background thread
            self.nameLabel.text = product.name // ❌ UIKit update off main thread!
            self.priceLabel.text = product.price
        }
    }
}
```

FIX

```
// Option A: Annotate function with @MainActor
@MainActor
func loadProduct(id: String) async throws {
    let product = try await fetchProduct(id: id) // suspends, but resumes on MainActor
    nameLabel.text = product.name // ✅ guaranteed main thread
}
```

```
// Option B: Annotate the class
@MainActor
class ProductVC: UIViewController { /* all methods on main actor */ }
```

HIGH Bug #7: Double-Resume of Continuation

Calling `continuation.resume` more than once crashes the app. This happens when a completion handler is called multiple times (e.g., a retry framework calling the callback on both success and timeout).

BROKEN CODE

```
func fetch() async throws → Data {
    try await withCheckedThrowingContinuation { continuation in
        client.request { data, error in
            if let error = error { continuation.resume(throwing: error) } // called on timeout
            if let data = data { continuation.resume(returning: data) } // ALSO called on suc
            // ❌ Both called if request completes after timeout fires
        }
    }
}
```

FIX

```
func fetch() async throws → Data {
    try await withCheckedThrowingContinuation { continuation in
        var resumed = false
        client.request { data, error in
            guard !resumed else { return } // ✅ guard against double-resume
            resumed = true
            if let error = error { continuation.resume(throwing: error) }
            else if let data = data { continuation.resume(returning: data) }
            else { continuation.resume(throwing: NSError(.badServerResponse)) }
        }
    }
}
```

HIGH Bug #8: Priority Inversion with Detached Tasks

Using `Task.detached` with `.background` priority for work that a high-priority task is waiting on. The high-priority task starves waiting for low-priority detached work to complete.

BROKEN CODE

```
@MainActor // high priority UI task
func showUserAvatar() async {
```

```

let image = await Task.detached(priority: .background) { // ❌ background!
    return await loadAndProcessImage() // high-priority UI waits on background task
}.value
avatarView.image = image // blocked for seconds due to priority inversion
}

```

FIX

```

@MainActor
func showUserAvatar() async {
    let image = await Task(priority: .userInitiated) { // ✅ inherit high priority
        return await loadAndProcessImage()
    }.value // OR simply: await loadAndProcessImage() directly
    avatarView.image = image
}

```

HIGH Bug #9: Forgetting Error Handling Causes Silent Failure

Using `Task { try await ... }` without handling errors. Errors thrown inside the task are silently discarded unless the caller awaits the task's value. The UI shows stale/empty state with no indication of failure.

BROKEN CODE

```

func onViewAppear() {
    Task {
        let data = try await fetchImportantData() // ❌ error silently discarded
        updateUI(data) // never called on error
    }
    // Error is never shown to user – empty screen with no feedback
}

```

FIX

```

func onViewAppear() {
    Task {
        do {
            let data = try await fetchImportantData()
            await updateUI(data)
        } catch {
            await showError(error) // ✅ always handle errors in top-level Task
        }
    }
}

```

MEDIUM**Bug #10: os_unfair_lock Misuse Across Await**

`os_unfair_lock` is tied to a specific thread (non-reentrant, owner-checks). Locking before an `await` and unlocking after means the lock is held across thread boundaries — the unlock may execute on a different thread than the lock, causing undefined behavior or crashes.

BROKEN CODE

```
var lock = os_unfair_lock()
func loadAndCache() async {
    os_unfair_lock_lock(&lock)           // locks on Thread A
    let data = await fetchData()         // ← Thread A is freed, resumes on Thread B!
    cache = data
    os_unfair_lock_unlock(&lock)         // ❌ unlocking on Thread B – undefined behavior
}
```

FIX — USE NSLOCK WHICH IS NOT THREAD-AFFINE

```
// Better fix: Use an actor – no locks needed at all
actor DataCache {
    var cache: Data?
    func loadAndCache() async {
        cache = await fetchData() // ✅ actor serializes access, no locks needed
    }
}
```

MEDIUM**Bug #11: TaskGroup Child Task Errors — First Error Cancels All**

In `withThrowingTaskGroup`, when one child task throws, the group is cancelled — but the error is only surfaced when you await the group. Unhandled, this can cause partially processed data being committed, leaving the app in an inconsistent state.

BROKEN CODE

```
try await withThrowingTaskGroup(of: Void.self) { group in
    for item in items {
        group.addTask { try await processAndSave(item) } // saves to DB
    }
    // ❌ If item[3] fails, items 0-2 are ALREADY saved to DB
    // Partial state committed – DB inconsistency
    try await group.waitForAll()
}
```

FIX — COLLECT FIRST, THEN SAVE

```
let results = try await withThrowingTaskGroup(of: ProcessedItem.self) { group in
    for item in items { group.addTask { try await process(item) } }
```

```

    var all = [ProcessedItem]()
    for try await result in group { all.append(result) }
    return all
}
// ✅ Only save if ALL succeeded – all-or-nothing transaction
try await saveAll(results)

```

MEDIUM

Bug #12: AsyncStream Continuation Held Past Actor Isolation

Storing an `AsyncStream.Continuation` in an actor and calling `yield` from a non-isolated callback context creates a data race — the continuation is `Sendable` but calling it concurrently from multiple threads can cause issues.

FIX

```

// ✅ AsyncStream.Continuation is Sendable – safe to call from any context
// But always store it in an actor to prevent concurrent yield calls
actor EventSource {
    private var continuation: AsyncStream<Event>.Continuation?

    func emit(_ event: Event) {
        continuation?.yield(event) // serialized by actor
    }
}

```

Architectural Patterns & Best Practices

Applying Swift concurrency effectively at the architecture level requires deliberate decisions about actor boundaries, data flow, and task lifetimes.

The Golden Rules



10 Golden Rules for Swift Concurrency

1. Never block a thread inside an async function — use `await` instead.
2. Re-validate actor state after every `await` inside an actor.
3. Always store Task references and cancel them when the owning object deinit.
4. Prefer actors for all shared mutable state over manual locks.
5. Use `@MainActor` on all ViewModels and UI-touching code.
6. Always handle errors in top-level `Task { }` closures — they don't propagate.
7. Use `withCheckedContinuation` during development, `withUnsafeContinuation` only after proving correctness.
8. Prefer `async let` for fixed parallel operations, `TaskGroup` for dynamic parallelism.
9. Make your model types `Sendable` (use structs by default).
10. Never use `os_unfair_lock`, mutex locks, or thread-local storage across `await` points.

Recommended Architecture Pattern

```
CleanArchitecture.swift  SWIFT

// — Layer 1: Domain Model (Sendable structs) —————
struct User: Sendable, Codable {
    let id: String
    let name: String
}

// — Layer 2: Repository (actor for shared state) —————
actor UserRepository {
    private var cache: [String: User] = [:]
```

```

    func getUser(id: String) async throws → User {
        if let cached = cache[id] { return cached }
        let user = try await UserAPIClient.fetch(id: id)
        cache[id] = user
        return user
    }
}

```

// — Layer 3: ViewModel (@MainActor for UI) —————

@MainActor

```

final class UserViewModel: ObservableObject {
    @Published private(set) var user: User?
    @Published private(set) var isLoading = false
    @Published private(set) var error: Error?

    private let repository: UserRepository
    private var loadTask: Task<Void, Never>?

    func loadUser(id: String) {
        loadTask?.cancel()
        loadTask = Task {
            isLoading = true
            error = nil
            do {
                user = try await repository.getUser(id: id)
            } catch is CancellationError {
                // intentional – don't show error
            } catch {
                self.error = error
            }
            isLoading = false
        }
    }

    deinit { loadTask?.cancel() }
}

```

// — Layer 4: View (SwiftUI – no async logic) —————

```

struct UserView: View {
    @StateObject private var vm = UserViewModel(repository: .shared)
    var body: some View {
        Group {
            if vm.isLoading { ProgressView() }
            else if let user = vm.user { Text(user.name) }
            else if let error = vm.error { Text(error.localizedDescription) }
        }
        .task { vm.loadUser(id: "current") } // auto-cancelled on disappear ✓
    }
}

```

```
}  
}  
}
```

Decision Guide: Which Tool to Use?

Situation	Use This	Why
2-3 independent async operations	<code>async let</code>	Clean, structured, compile-time count
Dynamic number of parallel tasks	<code>withTaskGroup</code>	Runtime dynamic, structured lifetime
Shared mutable state	<code>actor</code>	Compile-time race prevention
All UI code	<code>@MainActor</code>	Guaranteed main thread
Wrapping callback API	<code>withCheckedContinuation</code>	Safe, has runtime checks
Values over time (events, sensors)	<code>AsyncStream</code>	Structured, backpressure support
Fire-and-forget background work	<code>Task.detached</code>	No actor inheritance — be careful
Cross-actor database operations	Custom global actor	Domain-specific serialization
Limiting concurrent requests	TaskGroup + counter	Semaphore replacement
Testing async code	<code>async func test</code>	No XCTestExpectation needed

Quick Reference Cheat Sheet

A fast-access reference for all Swift concurrency APIs, syntax, and patterns.

Core Keywords & Types

```
CheatSheet.swift SWIFT
```

```
// — DECLARATIONS —
async func foo() → T           // async function
async func foo() throws → T    // async throwing function
actor MyActor { ... }          // actor type
@MainActor class MyVM { ... }   // main thread class
@GlobalActor actor MyGlobal { ... } // custom global actor
nonisolated func safeToCallAnywhere() // opt out of actor isolation
@Sendable () → Void            // sendable closure
struct Foo: Sendable { ... }    // sendable type
@TaskLocal static var key: T?    // task-local value
```

```
// — CALLING —
let x = await foo()             // await single
let x = try await foo()         // await throwing
async let a = foo(); let v = await a // concurrent binding
await MainActor.run { ... }     // run on main actor
```

```
// — TASKS —
Task { ... }                    // unstructured task (inherits context)
Task(priority: .background) { ... } // with priority
Task.detached { ... }           // detached task (no parent)
Task.detached(priority: .utility) { ... } // detached with priority
task.cancel()                   // request cancellation
task.value                      // await task result
Task.isCancelled                // check if cancelled
try Task.checkCancellation()     // throw if cancelled
await Task.yield()              // give up CPU voluntarily
try await Task.sleep(for: .seconds(2)) // non-blocking sleep
Task.currentPriority            // get current task priority
```

```
// — TASK GROUPS —
await withTaskGroup(of: T.self) { group in
    group.addTask { ... }
    for await result in group { ... }
```

```

}
try await withThrowingTaskGroup(of: T.self) { group in
    group.addTask { ... }
    try await group.waitForAll()
    try await group.next()           // get one result
    group.cancelAll()                // cancel all children
}

// — CONTINUATIONS —
await withCheckedContinuation { c in ... c.resume(returning: value) }
try await withCheckedThrowingContinuation { c in
    ...; c.resume(returning: v) // or c.resume(throwing: err)
}
withTaskCancellationHandler(operation: { ... }, onCancel: { ... })

// — ASYNC SEQUENCE —
for await item in asyncSequence { ... }
for try await item in throwingAsyncSequence { ... }
AsyncStream<T> { continuation in
    continuation.yield(value)           // emit value
    continuation.finish()               // end stream
    continuation.finish(throwing: err) // end with error
    continuation.onTermination = { _ in ... } // cleanup
}
AsyncThrowingStream<T, Error> { ... } // throwing variant

```

Migration Guide: GCD → Swift Concurrency

Old GCD Pattern	Swift Concurrency Equivalent
<code>DispatchQueue.global().async { }</code>	<code>Task { }</code>
<code>DispatchQueue.main.async { }</code>	<code>await MainActor.run { }</code> or <code>@MainActor func</code>
<code>DispatchQueue.sync { }</code>	No direct equivalent — use <code>actor</code> instead
<code>DispatchGroup</code>	<code>withTaskGroup</code>
<code>DispatchSemaphore</code>	<code>actor</code> or TaskGroup with counter
Completion handler	<code>async throws func</code> + <code>withCheckedContinuation</code>
<code>OperationQueue</code> with dependencies	<code>async let</code> or TaskGroup with <code>await</code>

`Thread.sleep`

`try await Task.sleep(for: .seconds(n))`

`NSLock` / `os_unfair_lock`

`actor` (preferred) or `NSLock` only for sync contexts

Combine publishers

`AsyncStream` / `AsyncSequence`

`DispatchQueue` with QoS

`Task(priority: .background)`

Summary: The Swift Concurrency Philosophy

- **Structured lifetimes** — tasks are always nested within their creating scope
- **Cooperative, not preemptive** — tasks yield voluntarily at `await` points, enabling efficient thread usage
- **Compile-time safety** — the `Sendable` system catches data races before runtime
- **Linear reasoning** — async code reads top-to-bottom like synchronous code
- **Automatic propagation** — cancellation, priority, and task-local values propagate through the task tree
- **Actor isolation** — all shared mutable state lives inside actors, accessed only through `await`

Swift Concurrency was introduced in Swift 5.5 (Xcode 13, iOS 15, macOS 12). Retroactive availability via `@available(macOS 10.15, iOS 13, *)` is supported for `async/await` but not the full structured concurrency model. Swift 6 (Xcode 16+) enables strict concurrency checking as a compile error. This document covers Swift 5.9–6.0.